

RoomieSync

Hostel Expense & Task Management Dashboard

Software Requirements Specification

Prepared in accordance with IEEE 830 / ISO/IEC/IEEE 29148 SRS guidelines

A Web-Based Platform for Managing Shared Hostel Expenses, Roommate Tasks and Settlements

Document Version	1.0
Document Status	Final – For Faculty Evaluation
Prepared For	GLA University
Course / Program	B.Tech – Semester 3
Prepared By	Deepanshu Thakur (22515000042)
Project Guide	Dr. Gautam Mukherjee
Date	16/08/2026

Table of Contents

Table of Contents	2
1. Introduction.....	4
1.1 Purpose.....	4
1.2 Document Conventions.....	4
1.3 Intended Audience and Reading Suggestions	4
1.4 Project Scope	4
1.5 References	4
2. Overall Description.....	6
2.1 Product Perspective	6
2.2 Product Features (Summary)	6
2.3 User Classes and Characteristics.....	6
2.4 Operating Environment	6
2.5 Design and Implementation Constraints.....	7
2.6 Assumptions and Dependencies.....	7
3. System Features (Functional Requirements)	8
3.1 Module: User Registration & Authentication	8
3.2 Module: Room / Group Management.....	8
3.3 Module: Expense & Settlement Management.....	9
3.4 Module: Task / Chore Management.....	9
3.5 Module: Room Admin Dashboard	10
3.6 Module: Notifications	10
4. External Interface Requirements.....	11
4.1 User Interfaces	11
4.2 Hardware Interfaces.....	11
4.3 Software Interfaces	11
4.4 Communication Interfaces.....	11
5. Use Case Summary	12
5.1 Primary Flow – Add Expense and Settle Balance (Roommate)	12
6. Non-Functional Requirements.....	13
6.1 Performance Requirements.....	13
6.2 Security Requirements.....	13
6.3 Usability	13
6.4 Reliability & Availability.....	13

6.5 Maintainability & Portability	13
6.6 Scalability	14
7. Software Testing Considerations	15
8. Proposed System Architecture	16
Presentation Layer	16
Application Layer.....	16
Data Layer.....	16
8.1 Suggested Technology Stack.....	16
9. Data Model Overview (for Database Planning)	17
9.1 Core Entities	17
9.2 Key Relationships	17
10. Future Enhancements (Out of Current Scope)	18
11. Glossary	19

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for RoomieSync, a web-based application that helps hostel and shared-flat residents manage shared expenses, split bills fairly, track settlements, and coordinate household chores/tasks within a room group. The document is intended to serve as the single source of truth for the development team, project guide, and evaluation committee regarding what the system will do, who will use it, and the constraints under which it must be built. It is written to be used for requirement analysis, UI/UX design, database and API planning, software testing, and final project documentation.

1.2 Document Conventions

Functional requirements are uniquely identified using the format FR-XX, and are grouped by module. Each requirement is written using the keyword “shall” to indicate a mandatory capability of the system, consistent with IEEE 830 / ISO/IEC/IEEE 29148 conventions. Priority levels used throughout this document are High (must be implemented for a working system), Medium (important but can be simplified if time is short) and Low (optional / future enhancement).

1.3 Intended Audience and Reading Suggestions

- Students / Development Team – to understand what to build, module by module.
- Project Guide / Faculty Evaluator – to assess completeness, correctness and feasibility of scope.
- UI/UX Designer – to design screens based on Section 4 and Section 3 workflows.
- Database / Backend Developer – to design schema and APIs based on Section 8 and Section 9.
- Software Tester – to derive test cases directly from the functional requirements in Section 3.

1.4 Project Scope

RoomieSync will allow hostel or shared-flat residents to form a room group, add shared expenses (rent, groceries, utilities, maintenance), split them equally, by exact amount, or by percentage, and automatically track each member's running balance. Members can record settlements, view a simplified list of who-owes-whom, and generate monthly expense summaries. In parallel, the platform lets members create and assign household chores/tasks, including recurring tasks such as cleaning or grocery runs, and track completion through a shared task board with fairness scoring.

The scope is deliberately kept realistic for a college-level team project. The system focuses on manual expense entry, rule-based bill splitting, and simple task assignment/rotation logic — it is not a banking, real-money-transfer, or property-management platform. Actual payment gateway integration, multi-currency support, native mobile applications, and AI-based expense categorization are explicitly out of scope for the current version and are listed as future enhancements in Section 10.

1.5 References

- IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications.
- ISO/IEC/IEEE 29148:2018 – Systems and Software Engineering – Requirements Engineering.
- React.js, Node.js (Express.js), MongoDB / MySQL official documentation.
- Project description document / synopsis submitted by the student team.

2. Overall Description

2.1 Product Perspective

RoomieSync is a new, self-contained, independent web application. It is not a modification of an existing property-management or payments system. It follows a standard three-tier web architecture (presentation, application/API, and data layer — detailed in Section 8), designed so that it can later be extended with payment-gateway or push-notification integrations as the project matures beyond the academic prototype stage.

2.2 Product Features (Summary)

- User registration, login and profile management.
- Room/flat group creation and invite-based membership.
- Shared expense logging with equal, exact-amount, or percentage-based splitting.
- Automatic balance calculation and simplified settlement suggestions.
- Household task/chore creation, assignment, rotation and completion tracking.
- Room Admin dashboard for budgets, member management and reports.
- Automated notifications for new expenses, due tasks and settlements.
- Basic analytical reports (monthly spend, category breakdown, task-completion trends).

2.3 User Classes and Characteristics

User Class	Description	Technical Characteristics
Visitor / Guest	General public viewing the landing page before signing up.	Basic web/smartphone literacy; no access to group data.
Roommate (Registered User)	Member of a hostel room/flat group who logs expenses and completes tasks.	Basic computer/smartphone literacy; primary end-user of the app.
Room Admin	Roommate who created or was assigned admin rights for a specific group.	Same as Roommate, plus group configuration and reporting access.

2.4 Operating Environment

- Client side: Any modern web browser (Chrome, Firefox, Edge, Safari) on desktop, laptop, tablet or smartphone with an active internet connection.
- Server side: Node.js/Express.js runtime hosted on a cloud platform (e.g. Render, Railway, AWS/Azure free tier) or a local server for demonstration.
- Database: MongoDB Atlas (cloud) or local MongoDB / MySQL instance.

2.5 Design and Implementation Constraints

- Must be developed and demonstrated within a single college semester by a small student team (typically 2–4 members).
- Must primarily use free / open-source technologies to keep the project cost at zero.
- Must be responsive and usable on both desktop and mobile screen sizes.
- Real payment-gateway settlement is treated as optional/future add-on; the system shall support manual settlement recording as the default method.
- The system assumes a single currency and a single institution/hostel context for the academic version.

2.6 Assumptions and Dependencies

- Users have basic familiarity with using web forms and everyday expense-splitting concepts.
- Each room group is small (typically 2–8 members), consistent with a hostel room or shared flat.
- Reliable internet connectivity is available at both client and server ends.
- Accuracy of balances depends on members entering expenses honestly and promptly; the system does not verify real-world receipts.

3. System Features (Functional Requirements)

This section lists the functional requirements of the system, grouped by module. Each requirement can be traced directly to a test case (see Section 7) and a screen (see Section 4).

3.1 Module: User Registration & Authentication

Description: Allows visitors to securely create accounts and log in, with role-based access.

ID	Requirement	Priority
FR-01	The system shall allow a new user to register using name, email, phone number and password.	High
FR-02	The system shall validate email format and enforce a minimum password strength on registration.	High
FR-03	The system shall authenticate registered users (Roommate, Room Admin) via email and password login.	High
FR-04	The system shall issue a secure session/token (e.g. JWT) upon successful login.	High
FR-05	The system shall allow a user to reset a forgotten password via a one-time email link/OTP.	Medium
FR-06	The system shall restrict access to module features based on the logged-in user's role (Roommate/Room Admin).	High
FR-07	The system shall allow a logged-in user to log out, invalidating the active session.	Medium

3.2 Module: Room / Group Management

Description: Allows users to form and manage hostel room or flat groups before logging expenses or tasks.

ID	Requirement	Priority
FR-08	The system shall allow a user to create a new hostel room/flat group and become its default Room Admin.	High
FR-09	The system shall allow a Room Admin to invite roommates to the group via a shareable link or invite code.	High
FR-10	The system shall allow a user to join an existing group using a valid invite code.	High
FR-11	The system shall allow a Room Admin to remove a roommate from the group or transfer admin rights.	Medium
FR-12	The system shall allow a user to belong to and switch between more than one active room group.	Medium
FR-13	The system shall display a group profile showing all current members, room number and move-in dates.	Low

3.3 Module: Expense & Settlement Management

Description: Core functionality for logging shared expenses, splitting bills, and tracking who-owes-whom.

ID	Requirement	Priority
FR-14	The system shall allow a roommate to add a shared expense with amount, category, date and paid-by member.	High
FR-15	The system shall allow an expense to be split equally, by exact amount, or by percentage among selected members.	High
FR-16	The system shall automatically calculate each member's running balance (owed / owing) within a group.	High
FR-17	The system shall allow a member to record a settlement/payment made to another member, updating balances.	High
FR-18	The system shall suggest a minimum set of settlement transactions needed to clear all group balances.	Medium
FR-19	The system shall allow a user to attach a receipt image or note to a recorded expense.	Low
FR-20	The system shall allow a user to filter and view expense history by category, member, or date range.	Medium
FR-21	The system shall allow a Room Admin to edit or delete an incorrectly entered expense, with a change log.	Medium

3.4 Module: Task / Chore Management

Description: Enables creation, assignment, rotation and completion tracking of household chores.

ID	Requirement	Priority
FR-22	The system shall allow a member to create a household task/chore with a title, description and due date.	High
FR-23	The system shall allow a task to be assigned to one member or rotated automatically among all members.	High
FR-24	The system shall allow an assigned member to mark a task as in-progress or completed.	High
FR-25	The system shall allow recurring tasks (daily/weekly/monthly) to be auto-regenerated after completion.	Medium
FR-26	The system shall display a shared task calendar/board visible to all members of the group.	High
FR-27	The system shall allow a member to mark a task as overdue and flag it to the group.	Medium
FR-28	The system shall maintain a task-completion score/streak per member for fairness tracking.	Low

3.5 Module: Room Admin Dashboard

Description: Group-level configuration, budgeting, oversight and reporting.

ID	Requirement	Priority
FR-29	The system shall allow a Room Admin to view a consolidated dashboard of group expenses and task completion.	High
FR-30	The system shall allow a Room Admin to set a monthly group budget and category-wise spending limits.	Medium
FR-31	The system shall allow a Room Admin to generate a monthly summary report of expenses and settlements.	Medium
FR-32	The system shall allow a Room Admin to post group-wide announcements (e.g. rent due date, rules).	Low

3.6 Module: Notifications

ID	Requirement	Priority
FR-33	The system shall send an email/in-app notification when a new expense is added to a group the user belongs to.	Medium
FR-34	The system shall send a reminder notification when a task assigned to the user is due within 24 hours.	Medium
FR-35	The system shall send a notification when another member records a settlement affecting the user's balance.	Medium
FR-36	The system shall send a weekly digest summarizing outstanding balances and pending tasks.	Low

4. External Interface Requirements

4.1 User Interfaces

- Landing / Home Page – project introduction, feature highlights, login/register call-to-action.
- Group Dashboard – balance summary, recent expenses, upcoming tasks, quick-add buttons.
- Expense Screen – add/edit expense form, split configuration, expense history and filters.
- Task Board – kanban-style or list view of chores by status (Pending/In-Progress/Completed).
- Room Admin Dashboard – manage members, set budgets, generate reports, post announcements.
- All screens shall follow a consistent, clean, responsive layout (see Section 6.3) using a warm, homely colour palette (soft teal with warm accent tones).

4.2 Hardware Interfaces

No specialized hardware is required. The system runs on standard client devices (desktop/laptop/tablet/smartphone) with a working internet connection, and on any commodity server/cloud instance capable of running Node.js and MongoDB/MySQL.

4.3 Software Interfaces

Component	Technology
Frontend	React.js (with React Router, Axios, and a component library such as Bootstrap/Tailwind/MUI)
Backend / API	Node.js with Express.js – RESTful JSON APIs
Database	MongoDB (Mongoose ODM) or MySQL – stores users, groups, expenses, tasks and settlements
Authentication	JWT (JSON Web Tokens) with bcrypt password hashing
Email Service	SMTP service (e.g. Nodemailer with Gmail/SendGrid) for notifications
Hosting (suggested)	Frontend – Vercel/Netlify; Backend/API – Render/Railway; Database – MongoDB Atlas

4.4 Communication Interfaces

- All client–server communication shall occur over HTTPS using RESTful JSON APIs.
- Outbound email notifications shall be sent via the SMTP protocol through the configured email service.
- (Future phase) Payment settlement may be integrated via a third-party payment gateway API.

5. Use Case Summary

The table below summarizes the primary use cases per actor. Detailed step-by-step use-case descriptions (preconditions, main flow, alternate flow) can be expanded from this list during design if required by the evaluation format.

Actor	Use Case	Brief Description
Roommate	Register / Login	Create an account or sign in to access the group dashboard.
Roommate	Create / Join Group	Start a new room group or join an existing one via invite code.
Roommate	Add Expense	Log a shared expense and split it among selected members.
Roommate	Settle Balance	Record a payment made to another member to clear a balance.
Roommate	Manage Tasks	Create, accept, or complete an assigned household task.
Room Admin	Manage Group Members	Invite, remove, or promote roommates within a group.
Room Admin	Set Budget	Define a monthly group budget and category limits.
Room Admin	View Reports	Review monthly expense summaries and task-completion trends.

5.1 Primary Flow – Add Expense and Settle Balance (Roommate)

1) User logs in → 2) Selects their active room group → 3) Adds a new expense with amount, category and split method → 4) System calculates each member's updated share and balance → 5) Result is reflected on the group dashboard and other members are notified → 6) When a member repays another, they record a settlement, and the system updates both balances accordingly.

6. Non-Functional Requirements

6.1 Performance Requirements

ID	Requirement	Priority
NFR-01	The system shall load any core page within 3 seconds under normal load on a standard broadband connection.	High
NFR-02	The system shall support at least 50–100 concurrent users without noticeable performance degradation (adequate for a college demo/pilot).	Medium
NFR-03	A balance recalculation shall be completed and reflected within 2 seconds of an expense or settlement being added.	Medium

6.2 Security Requirements

ID	Requirement	Priority
NFR-04	Passwords shall be stored using a one-way hashing algorithm (e.g. bcrypt); plain-text passwords shall never be stored.	High
NFR-05	All API endpoints that access personal, financial or group data shall require a valid authentication token.	High
NFR-06	Role-based authorization shall ensure a standard Roommate cannot access Room-Admin-only endpoints of another group.	High
NFR-07	All data in transit shall be encrypted using HTTPS/TLS.	High
NFR-08	User input shall be validated and sanitized on both client and server to prevent injection attacks.	High

6.3 Usability

- The interface shall be simple enough for a first-time user to add an expense in under 5 clicks/taps.
- The system shall be fully responsive across desktop, tablet and mobile screen sizes.
- Error and success messages shall be clear, specific, and shown close to the relevant form field/action.

6.4 Reliability & Availability

- The system shall target 99% uptime during the demonstration/evaluation period.
- Expense, settlement and task data shall not be lost due to a single server restart (persisted in the database, not in memory).

6.5 Maintainability & Portability

- The codebase shall follow a modular MVC-style structure (routes/controllers/models on the backend, reusable components on the frontend) to ease future maintenance.

- The system shall run on any OS capable of hosting Node.js and MongoDB/MySQL (Windows, Linux, macOS), and shall be accessible from any modern browser without requiring a native app install.

6.6 Scalability

Although scoped for small hostel/flat groups in the academic version, the data model and API design shall avoid hard-coded single-group assumptions where reasonably possible, so that a user belonging to multiple groups, larger flats, or future payment-gateway integration can be added without a full redesign.

7. Software Testing Considerations

Each functional requirement in Section 3 shall map to at least one test case. The table below gives representative examples to guide the team's test plan; the full test case document can be maintained separately (e.g. in a spreadsheet) using this format.

Test ID	Req.	Test Scenario	Expected Result
TC-01	FR-15	Add an expense split equally among 4 members	Each member's balance updated by the correct proportional share.
TC-02	FR-18	Request a settlement suggestion for a group with circular debts	System returns the minimum number of transactions to settle all balances.
TC-03	FR-24	Mark an assigned task as completed	Task status changes to Completed and streak score is updated.
TC-04	FR-06	Standard roommate attempts to access another group's Admin page	Access denied (403) / redirected to login.
TC-05	FR-02	Register with a weak password	Registration blocked with validation message.

Recommended testing types: Unit testing (backend controllers and validators using a framework such as Jest/Mocha), Integration testing (API endpoints with Postman/Jest+Supertest), Balance-calculation testing (verifying split and settlement arithmetic against manually computed expected values), and Manual UI/UX testing across at least two browsers and one mobile viewport before final submission.

8. Proposed System Architecture

Given the team's familiarity and the project's realistic academic scope, a MERN-style web stack (MongoDB, Express.js, React.js, Node.js) is recommended. This keeps the system well documented, free to host on student-friendly free tiers, and maps cleanly onto the module breakdown in Section 3.

Presentation Layer

React.js SPA – Roommate / Room Admin UI

Application Layer

Node.js + Express.js REST API – Auth, Groups, Expenses, Tasks, Notifications

Data Layer

MongoDB – Users, Groups, Expenses, Settlements, Tasks, Notifications

Figure 8.1: Three-tier architecture of RoomieSync.

8.1 Suggested Technology Stack

Layer	Technology
Frontend	React.js, React Router, Axios, Bootstrap / Tailwind CSS
Backend (Core API)	Node.js, Express.js, JWT, bcrypt, Nodemailer
Database	MongoDB with Mongoose (or MySQL)
Version Control	Git & GitHub
Testing	Postman (API testing), Jest (unit testing)
Deployment (suggested)	Vercel/Netlify (frontend), Render/Railway (backend), MongoDB Atlas (database)

9. Data Model Overview (for Database Planning)

The following core entities and relationships are proposed as a starting point for schema design. This is intentionally kept minimal and realistic for a student project; fields can be extended during implementation.

9.1 Core Entities

Entity	Key Attributes
User	_id, name, email, passwordHash, phone, createdAt
Group (Room)	_id, name, inviteCode, roomNumber, createdBy, createdAt
Membership	_id, groupId, userId, role (roommate/admin), joinedAt
Expense	_id, groupId, paidBy, amount, category, description, splitType, splitDetails, date, receiptUrl
Settlement	_id, groupId, fromUserId, toUserId, amount, date, note
Task	_id, groupId, title, description, assignedTo, dueDate, status, recurrence, streakPoints
Notification	_id, userId, message, type, isRead, createdAt

9.2 Key Relationships

- A User can belong to many Groups, and a Group has many Users (many-to-many via Membership).
- A Group has many Expenses; each Expense is paid by one member and split among several members.
- A Settlement records a payment from one member to another within a Group (many-to-one on both sides).
- A Group has many Tasks; each Task is assigned to one member at a time, with optional recurrence.
- A User can receive many Notifications (one-to-many).

10. Future Enhancements (Out of Current Scope)

- Integration with a real payment gateway (UPI/Stripe/Razorpay) for direct in-app settlement.
- Push notifications via a mobile app (Android/iOS) instead of email/in-app only.
- AI-based receipt scanning and automatic expense categorization.
- Multi-currency and multi-hostel/institution support.
- Gamified fairness leaderboard for task completion across groups.
- Expense analytics with predictive monthly budgeting suggestions.

11. Glossary

Term	Definition
SRS	Software Requirements Specification
API	Application Programming Interface
REST	Representational State Transfer – a web API architecture style
JWT	JSON Web Token – used for stateless authentication
MERN	MongoDB, Express.js, React.js, Node.js – a full JavaScript web stack
CRUD	Create, Read, Update, Delete – basic data operations
OTP	One-Time Password
UI/UX	User Interface / User Experience
Settlement	A recorded payment between two members that reduces an outstanding balance

End of Document – RoomieSync SRS v1.0